

§5 神经网络 Neural Networks

§5.1 术语与简介 Terminology

Def 5.1.1 神经网络 Neural Networks (NN)

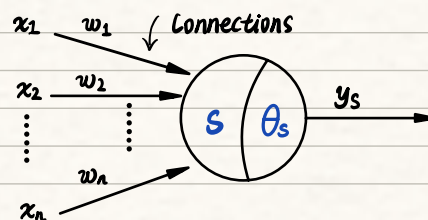
神经网络是具有适应性的简单单元组成的广泛互联的并行网络,其组织能够模拟生物神经系统对真实世界作出的交互反应;机器学习中所讨论的神经网络一般指神经网络学习,指利用神经网络技术进行监督/非监督/强化学习任务

Def 5.1.2 McCulloch-Pitts 神经元模型 M-P Neuron Model

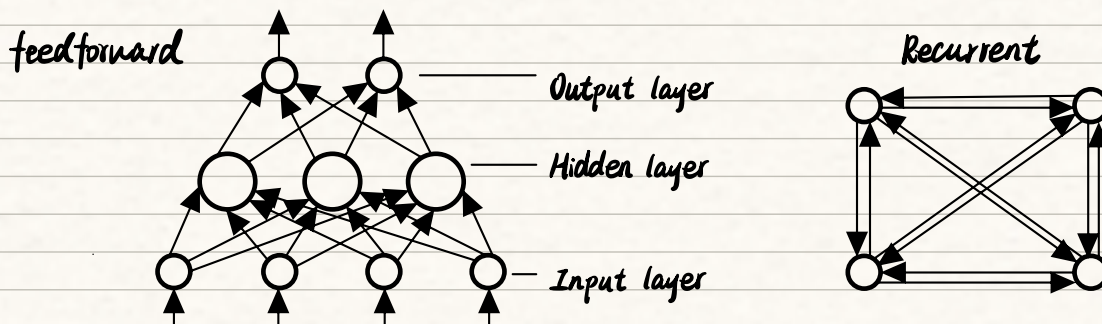
神经网络中最基本的成分为神经元 Neurons/nodes 模型;在生物神经网络中,多个神经元相互连接,并互相发送化学物质引起神经元内部电位改变;当某神经元内的电位超过一个“阈值” threshold,则其被激活并向其它相连神经元发送化学物质(信号)。

1943年, McCulloch & Pitts 将上述情形抽象为如下模型,称为 M-P 神经元模型并沿用至今,在下列模型中,目标神经元 S 接收来自与其相连的几个神经元的信号 $(x_1, \dots, x_n)$, 这些信号通过加权处理后与目标神经元的阈值 θ_s threshold / bias 比较,并由激活函数 ψ activation function 处理后输出为由目标神经元发出的信号 y_s :

$$y_s = \psi(\sum_{i=1}^n w_i x_i - \theta_s) \quad (+\theta_s \text{ in notes})$$



一个神经网络通常由多层神经元相互连接构成,按照每个神经元的功能将它们分类为输入神经元 Input nodes: 其输入=输出,人为指定;输出神经元 Output nodes: 输出信号由人工解读;隐藏神经元 Hidden nodes: 其输入为前层神经元的输出;输出为后层神经元的输入,不与人工交互;两个神经元之间的“连接”关系称为 Connection;对于单个神经元,输入连接的个数称为 fan-in;输出连接的个数称为 fan-out;确定一个神经网络应明确以下的三个因素: ① network architecture 即节点与连接关系; ② updating rules 即 activation function & threshold for each node; ③ learning rules 即计算连接权重的算法. 按照网络中是否存在回路可将其分类为网络中无回路的前馈网络 feedforward network 与网络中有回路的循环网络/反馈网络 Recurrent network



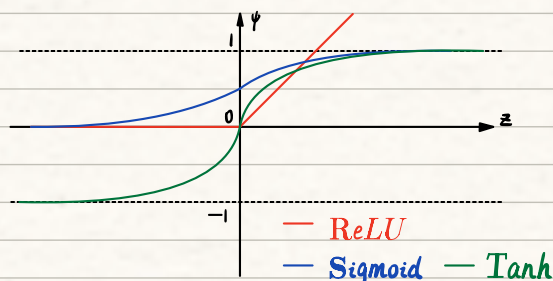
Def 5.1.3 常用激活函数 ψ Common Activation Functions

- | | | | |
|-----------|---|-------------|--|
| ① ReLU | $\psi(z) = z \cdot 1_{[0, +\infty)}(z)$ | ④ Signum | $\psi(z) = 1_{[0, +\infty)}(z) - 1_{(-\infty, 0)}(z) \triangleq \text{Sgn}(z)$ |
| ② Sigmoid | $\psi(z) = \frac{1}{1+e^{-z}}$ | ⑤ Heaviside | $\psi(z) = 1_{[0, +\infty)}(z)$ |
| ③ Tanh | $\psi(z) = \tanh(z)$ | | |

关于激活函数的选择:

- ① 分类问题: Signum, Heaviside, Sigmoid ✓ linear functions ✗ $\rightarrow$ 线性不可分 (e.g. XOR)
- ② 深度学习: ReLU ✓ Sigmoid, Tanh sometimes ✗ $\rightarrow$ 梯度消失 Vanishing Gradient

下面为 ReLU, Sigmoid, Tanh 的图像

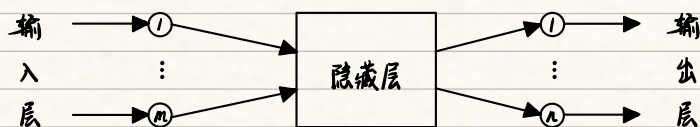


Sigmoid $\psi(z) = \frac{1}{1+e^{-z}}$ $\psi'(z) = \psi(z)(1-\psi(z))$

Tanh $\psi(z) = \tanh(z)$ $\psi'(z) = 1 - \tanh^2(z)$

Def 5.1.4 神经网络与监督学习

记模型的特征空间与标签空间为 F^m, L^n ; 给定训练集 $TR = \{(x_i, y_i) \in F^m \times L^n\}$, 由神经网络的 learning rules, 结合 TR 训练各个节点间的权重与阈值, 从而得到 $h: F^m \rightarrow L^n$; 一般来说, 该神经网络应有 m 个输入节点与 n 个输出节点, 记训练出的函数为 $h(x; w, \theta)$



step 1 构建网络, 制定学习规则

step 2 由 TR 训练权重与阈值, 得到 h

step 3 利用 $h(x; w, \theta)$ 预测/分类

注意, 通常可将阈值视为前一层中固定输出 -1 的哑节点 dummy node 所对应的权重

§ 5.2 梯度下降与逆误差传播算法 Gradient Descent & Error BackPropagation Algorithm

Algo 5.2.1 梯度下降法 Gradient Descent

梯度下降法是最常用的寻找可微多元函数 $f: \mathbb{R}^m \rightarrow \mathbb{R}$ 局部最小的优化算法, 其原理为: $f(\bar{x})$ 处, f 沿 $-\nabla f(\bar{x})$ 方向减小速度最快, 记 $\alpha > 0$ 通过下列迭代使初始点 $\bar{x}_0$ 收敛到使 f 取局部最小的极值点 $\bar{x}$

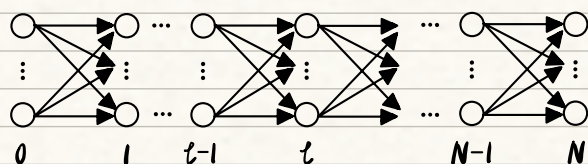
$$\bar{x}_{n+1} = \bar{x}_n - \alpha \nabla f(\bar{x}_n) \quad \text{if} \quad \|\nabla f(\bar{x}_n)\| > \varepsilon$$

Def 5.2.2 逆误差传播算法 Error BackPropagation Algorithm (BP)

逆误差传播算法是用于确定神经网络权重与阈值的最成功的学习规则之一, 其核心思路是: 将神经网络在训练集下的均方误差作为优化函数, 从而利用梯度下降法更新权重与阈值直至收敛, 我们记:

- $TR = \{(x, y) \in F^m \times L^n\} \subseteq \mathbb{R}(\psi)^m \times \mathbb{R}(\psi)^n$ 为标准化*后的训练集, $\mathbb{R}(\psi)$ 为激活函数值域
- $\forall (x, y) \in TR: E_{xy}(w, \theta) = \|h(x; w, \theta) - y\|^2$ 为给定训练数据 (x, y) 下的均方误差
- $E_{TR}(w, \theta) = |TR|^{-1} \cdot \sum_{(x, y) \in TR} E_{xy}$ 为给定训练集上的累积误差

考虑一个有如下结构的 $(N+1)$ 层前馈网络, 其中第 0 层与第 N 层分别为输入层与输出层, 记:



- w_{ij}^l — 前一层 $(l-1)$ 第 i 个节点指向 l 层第 j 个节点的权重
- z_j^l — l 层第 j 个节点的总体输入 $z_j^l = \sum_{i=1}^n w_{ij}^l s_i^{l-1} - \theta_j^l$
- s_j^l — l 层第 j 个节点的输出 $s_j^l = \psi(z_j^l)$
- y_j — 输出层第 j 个节点的输出 $y_j = s_j^N$

现任给 $(x, y) \in TR$, 考虑最小化 $E_{xy}(w, \theta) = \|h(x; w, \theta) - y\|^2 = \sum_{j=1}^n (s_j^N - y_j)^2$, 我们取 Sigmoid 激活函数, $\psi(z) = \psi(z)(1-\psi(z))$; 对于输出层: $w_{ij}^N \rightarrow z_j^N \rightarrow s_j^N \rightarrow E_{xy}$

$$\begin{aligned} \partial_{(w_{ij}^N)} E_{xy} &= \partial_{(s_j^N)} E_{xy} \cdot \partial_{(z_j^N)} s_j^N \cdot \partial_{(w_{ij}^N)} z_j^N = 2(s_j^N - y_j) \cdot \partial_{(z_j^N)} \psi(z_j^N) \cdot s_i^{N-1} \\ &= 2(s_j^N - y_j) \cdot \psi(z_j^N)(1-\psi(z_j^N)) \cdot s_i^{N-1} = 2(s_j^N - y_j) s_j^N (1-s_j^N) s_i^{N-1} = s_i^{N-1} \Delta_j^N \end{aligned}$$

由阈值视为前一层中固定输出 -1 的哑节点 $\partial_{(\theta_j^N)} E_{xy} = (-1) \Delta_j^N$

对于 $N-1$ 层: $w_{ij}^{N-1} \rightarrow z_j^{N-1} \rightarrow s_j^{N-1} \rightarrow z_k^N \rightarrow s_k^N \rightarrow Exy$

$$\begin{aligned}\partial_{(w_{ij}^{N-1})} Exy &= \sum_k \partial_{(s_k^N)} Exy \cdot \partial_{(z_k^N)} s_k^N \cdot \partial_{(s_j^{N-1})} z_k^N \cdot \partial_{(z_j^{N-1})} s_j^{N-1} \cdot \partial_{(w_{ij}^{N-1})} z_j^{N-1} \\ &= \sum_k \Delta_k^N \cdot w_{jk}^N \cdot s_j^{N-1} (1 - s_j^{N-1}) \cdot s_k^{N-2} = s_i^{N-2} s_j^{N-1} (1 - s_j^{N-1}) \sum_k \Delta_k^N \cdot w_{jk}^N = s_i^{N-2} \Delta_j^{N-1}\end{aligned}$$

记 $\Delta_j^{N-l} = s_j^{N-l} (1 - s_j^{N-l}) \sum_k \Delta_k^{N-l+1} w_{jk}^{N-l+1}$, $\Delta_j^N = 2(s_j^N - y_j) s_j^N (1 - s_j^N)$, 则 $\partial_{(s_j^{N-l})} Exy = \sum_k \Delta_k^{N-l+1} \cdot w_{jk}^{N-l+1}$

对 $l=1, \dots, N-1$ 均成立, 由上述讨论, 已证 $l=1$ 成立;

假设 $\partial_{(s_j^{N-l})} Exy = \sum_k \Delta_k^{N-l+1} \cdot w_{jk}^{N-l+1}$ 对 $l \geq 1$ 成立, 则对 $l+1$: $s_j^{N-(l+1)} \rightarrow z_p^{N-l} \rightarrow s_p^{N-l} \rightarrow Exy$

$$\begin{aligned}\partial_{(s_j^{N-(l+1)})} Exy &= \sum_p \partial_{(s_p^{N-l})} Exy \cdot \partial_{(z_p^{N-l})} s_p^{N-l} \cdot \partial_{(s_j^{N-(l+1)})} z_p^{N-l} \\ &= \sum_p \sum_k \Delta_k^{N-l+1} \cdot w_{pk}^{N-l+1} \cdot s_p^{N-l} (1 - s_p^{N-l}) \cdot w_{jp}^{N-l} = \sum_p \Delta_p^{N-l} \cdot w_{jp}^{N-l} \text{ 成立}\end{aligned}$$

于是 $N-l$ 层: $w_{ij}^{N-l} \rightarrow z_j^{N-l} \rightarrow s_j^{N-l} \rightarrow Exy$

$$\begin{aligned}\partial_{(w_{ij}^{N-l})} Exy &= \partial_{(s_j^{N-l})} Exy \cdot \partial_{(z_j^{N-l})} s_j^{N-l} \cdot \partial_{(w_{ij}^{N-l})} z_j^{N-l} = \sum_k \Delta_k^{N-l+1} \cdot w_{jk}^{N-l+1} \cdot s_j^{N-l} (1 - s_j^{N-l}) \cdot s_i^{N-l-1} \\ &= s_i^{N-l-1} s_j^{N-l} (1 - s_j^{N-l}) \sum_k \Delta_k^{N-l+1} \cdot w_{jk}^{N-l+1} = s_i^{N-l-1} \Delta_j^{N-l}\end{aligned}$$

$$\partial_{(s_j^{N-l})} Exy = (-1) \Delta_j^{N-l} \quad \text{s.t.} \quad \Delta_j^{N-l} = s_j^{N-l} (1 - s_j^{N-l}) \sum_k \Delta_k^{N-l+1} w_{jk}^{N-l+1}, \quad \Delta_j^N = 2(s_j^N - y_j) s_j^N (1 - s_j^N)$$

得到 **Sigmoid** 激活函数下 $(N+1)$ 层前馈网络对单-训练数据 $(x, y) \in TR$ 的BP逆推公式, 总结如下

Algo 5.2.3 基于Sigmoid的BP算法前传公式

给定 $(x, y) \in TR$ 与 $w, \theta \leftarrow w_t, \theta_t$; Sigmoid 作激活函数, 则 w, θ 的一次更新基于如下储存格式与步骤:

储存格式: 我们将每次的 (w, θ) 储存为如下的数据结构 $\{(w_t, \theta_t)\}_{t=1}^N$, 其中 l 为网络的 l 层, $l=0 \sim N$, 第 $1 \sim N-1$ 为隐藏层, 记:

$$w^l \triangleq (w_{ij}^l) = \begin{bmatrix} w_{1,1}^l & \dots & w_{1,n_l}^l \\ \vdots & \ddots & \vdots \\ w_{n_{l-1},1}^l & \dots & w_{n_{l-1},n_l}^l \end{bmatrix} \quad \begin{aligned} \theta^l &= [\theta_1^l \dots \theta_{n_l}^l]^T & l=1 \sim N \\ s^l &= [s_1^l \dots s_{n_l}^l]^T & l=0 \sim N \\ \Delta^l &= [\Delta_1^l \dots \Delta_{n_l}^l]^T & l=1 \sim N \end{aligned}$$

Step 1 正传: 正向计算 $h(x; w_t, \theta_t)$ 并记录下各层输出 $s^l, s^0 = x^T$

$$\text{for } l=1, \dots, N: s^l = \psi^T((s^{l-1})^T w^l * (\theta^l)^T) \quad (\psi^T \text{ ele-wise + 转置, } +(\theta^l)^T \text{ if } +\theta)$$

Step 2 逆传计算 Δ_j : 记 $x \odot y = (x_i y_i)$ ele-wise product

$$\text{for } l=0, \dots, N-1: \Delta^{N-l} = \begin{cases} s^N \odot (1 - s^N) \odot 2(s^N - y) & \text{if } l=0 \\ s^{N-l} \odot (1 - s^{N-l}) \odot (w^{N-(l-1)} \Delta^{N-(l-1)}) & \text{if } l=1, \dots, N-1 \end{cases}$$

Step 3 更新 (w, θ) : 记 $\Pi = 1_{w_{ij} \neq 0}$ 表示 $(l-1)$ 层的 i 节点与 l 层的 j 节点有连接

$$\begin{aligned} \text{for } l=1, \dots, N: \quad w^l(t+1) &\leftarrow w^l - \alpha \{s^{l-1} (\Delta^l)^T\} \odot \Pi \\ \theta^l(t+1) &\leftarrow \theta^l - \alpha (-1^*) \Delta^l \quad (* \text{ 用 } 1 \text{ 代替 } -1 \text{ if } +\theta) \end{aligned}$$

Algo 5.2.4 基于Sigmoid的标准BP算法 Standard BP

给定标准化训练集 $TR^* = \{(x, y) \in F^m \times L^1\} \subseteq (0, 1)^m \times (0, 1)^1$, 学习率 learning rate $d > 0$ 与初始化参数 $(w_0, \theta_0) \triangleq \{(w_0^l, \theta_0^l) : l = 1 \sim N\}$, 激活函数为 Sigmoid $\psi(z)$, 则标准 BP 算法的总流程为:
(取 $y_s = \psi(\sum_{i=1}^m w_i x_i - \theta_s)$, i.e. $-\theta$)

Step 1 $w, \theta \leftarrow w_0, \theta_0$; $E_{TR} \leftarrow |TR|^{-1} \cdot \sum_{(x, y) \in TR} E_{xy}(w, \theta)$

Step 2 for $(x, y) \in TR$: 进行 5.2.3 的 step 1~3, 从 $w^l, \theta^l = w_0^l, \theta_0^l$ 开始

for $l = 1, \dots, N$: $s^0 = x^T$, $s^l = \psi^T((s^{l-1})^T w^l - (\theta^l)^T)$

for $l = 0, \dots, N-1$: $\Delta^{N-l} = \begin{cases} s^N - (1 - s^N) \div 2(s^N - y) & \text{if } l = 0 \\ s^{N-l} \div (1 - s^{N-l}) \div (w^{N-(l-1)} \Delta^{N-(l-1)}) & \text{if } l = 1, \dots, N-1 \end{cases}$

for $l = 1, \dots, N$: $w^l \leftarrow w^l - d[s^{l-1}(\Delta^l)^T] \div \Pi$
 $\theta^l \leftarrow \theta^l - d(-1^*) \Delta^l$

Step 3 $E_{TR} \leftarrow |TR|^{-1} \cdot \sum_{(x, y) \in TR} E_{xy}(w, \theta)$; $E_{TR} < \varepsilon$: end, $E_{TR} \geq \varepsilon$: repeat Step 2

Remark (*) 关于标准化: 有助于搜索到 (w, θ) 的全局最优; 此外, 利用 Sigmoid 激活函数, 神经网络的输出必然介于 $(0, 1)$; 标准化也是为了方便后续的预测

(*) 关于 (w_0, θ_0) 的选则: 若取 Sigmoid / Tanh 为激活函数, 业内常用的初始值取法为 Xavier Uniform 即取 $w_0^l, \theta_0^l \sim UNI(-\sqrt{\frac{6}{n_{l-1} + n_l}}, \sqrt{\frac{6}{n_{l-1} + n_l}})$ 或 Xavier Normal $w_0^l, \theta_0^l \sim N(0, \sqrt{\frac{2}{n_{l-1} + n_l}})$

已经证明: 只要一个包含足够多神经元的隐层, 前馈网络就能以任意精度逼近任意复杂度的函数 (Hornik, 1989)
因此, BP 神经网络也常常遭遇过拟合, 有两种策略可以在一定程度上缓解这一问题: 早停 early stopping 与正则化 regularization (周志华 p106)

Def 5.2.5 累积BP算法 Accumulated BP

注意在标准 BP 算法中, 每次更新均仅针对单一的训练样例 $(x, y) \in TR$, 即公式是由最小化 E_{xy} 而非最小化 E_{TR} 得出的; 但如果将 5.2.2 推导中的 E_{xy} 替换为 E_{TR} , 从而推出一个直接最小化 E_{TR} 的递推公式后迭代, 每次迭代更新都直接针对 TR 整体, 所得算法即为累积 BP 算法

对比来说, 标准 BP 算法更新更频繁, 且由于对不同样例更新的效果可能出现“抵消”现象, 往往要进行更多次的迭代; 而累积 BP 算法直接最小化 E_{TR} , 其更新频率低得多, 但当 TR 较大时, 累积误差的下降将会非常缓慢, 此时用标准 BP 算法往往效果更好